

GAMECA v0.4.39

Installation, Quick-Start & Output Reference

Anmol Dash

June 27, 2026

Contents

1	What is GAMECA?	3
2	System requirements	4
3	Download	4
4	Installation	4
5	First launch and automatic setup	4
6	The interface	5
7	Connecting to an HPC cluster	5
8	Running an analysis	5
8.1	Check the locus count first	5
8.2	Single family — on the cluster	5
8.3	Single family — locally	6
8.4	Many families — batch submission	6
9	Output reference	6
9.1	Sequences and statistics (the inputs to everything)	7
9.2	Clustering: the master table	8
9.3	Alignment and consensus	8
9.4	Motif analysis (<code>motif_analysis/</code>)	9
9.5	Enrichment (<code>enrichment_results/</code>)	9
9.6	TFBS analysis (<code>05_tfbs/</code>)	9
9.7	GO annotation (<code>go_annotations/</code>)	10
9.8	Primers (<code>06_primers/</code>)	10
9.9	The visual dashboard (<code>07_visualizations/</code>)	10
9.10	Stage 11 standout analyses (<code>reports/</code>)	11
9.11	The compiled PDF report	14
9.12	Logs, caches, and intermediate files	14
10	Updates	15

1 What is GAMECA?

GAMECA (Gene Alignment, Motif, Expression and Clustering Analysis) is a native macOS desktop application for the comprehensive downstream characterisation of transposable-element (TE) families. It ships as a `.dmg` installer containing a Tauri/React frontend and a bundled Python analysis backend. Starting from nothing more than a RepeatMasker family name and a genome assembly code, GAMECA runs the following stages:

1. **Data prep** — fetch all annotated genomic copies from UCSC RepeatMasker or Dfam, then extract sequences (from a local genome FASTA or via the UCSC API).
2. **Clustering** — group copies by k -mer profile (SVD $\rightarrow$ UMAP/t-SNE $\rightarrow$ HDBSCAN).
3. **Alignment** — per-cluster MAFFT alignment, CIALign cleaning, and majority-vote consensus sequences.
4. **Motif analysis** — scan every copy for JASPAR transcription-factor binding motifs.
5. **Enrichment** — Fisher’s exact test for motifs over-represented in each cluster.
6. **TFBS analysis** — count and visualise transcription-factor binding sites per cluster and per strand.
7. **GO annotation** — map enriched motifs to their transcription factors and Gene Ontology terms.
8. **Expression** — per-cluster expression boxplots and differential-expression tests (when an expression matrix is provided).
9. **Primer design** — family- and cluster-specific PCR primers with genome-wide specificity checking.
10. **Stage 11 standout analyses** — phylogenetics, CRISPR gRNA off-target scoring, 3’ transduction detection, antisense promoter scanning, CTCF/TAD overlap, epigenetic overlay, ortholog insertion presence, multi-assembly liftover, ColabFold consensus ORF structure prediction, repeat landscape, LTR structural classification, subfamily resolution, and differential expression between clusters.

Analyses run either **locally** on your Mac or are submitted as independent **batch jobs** on an LSF or Slurm HPC cluster over SSH. No terminal knowledge is required for typical use.

2 System requirements

Component	Requirement
macOS	12 Monterey or later (Apple Silicon <i>or</i> Intel)
Disk	~3 GB (app + Python environment + first ColabFold model weights)
RAM	8 GB minimum; 16 GB or more recommended for large families
Internet	Required for UCSC/Dfam sequence fetching, first-run package install, JASPAR/GO lookups, and HPC SSH
HPC cluster	Optional; LSF (<code>bsub</code>) or Slurm (<code>sbatch</code>) reachable by SSH from your Mac

3 Download

Installers are published on the GitHub Releases page:

<https://github.com/anmol-dash/te-scraper-and-analysis/releases>

Download the `.dmg` that matches your Mac’s processor:

Architecture	File to download
Apple Silicon (M1/M2/M3/M4)	<code>GAMECA_0.4.39_aarch64.dmg</code>
Intel	<code>GAMECA_0.4.39_x64.dmg</code>

If unsure which processor your Mac has, click the Apple menu → **About This Mac** and read the “Chip” or “Processor” line.

4 Installation

1. Double-click the downloaded `.dmg` to mount it.
2. Drag **GAMECA.app** into your **Applications** folder.
3. Eject the disk image (`Cmd-E`).
4. **First launch only — Gatekeeper:** because GAMECA is distributed outside the Mac App Store, macOS blocks the first open. **Right-click** (or Control-click) **GAMECA.app** → **Open** → **Open**. This prompt appears once only.

5 First launch and automatic setup

On first launch GAMECA performs a one-time setup (~2–3 minutes):

1. The bundled Python sidecar starts and sends a **ping** handshake to the frontend; a spinner shows until it completes.
2. The app checks for a Python virtual environment at `$HOME/gameca_venv`. If absent, it creates one and installs all dependencies from the bundled `requirements.txt` (NumPy,

pandas, scikit-learn, UMAP, HDBSCAN, BioPython, MAFFT bindings, CAlign, MOODS, colabfold[alphafold], and more).

3. A progress bar and log stream in the **Log** panel. Do not close the app during this step.
4. When the status indicator turns green the app is ready.

Note: ColabFold model weights (~4 GB) are *not* downloaded here; they are fetched the first time a structure-prediction job runs and cached in `$HOME/.cache/colabfold`.

6 The interface

Sidebar (left). Every pipeline step, grouped into *Data prep*, *Core analysis*, *Enrichment*, *Primer design*, *Full pipeline*, and *Results*. Click a step to open its configuration panel.

Configuration panel (centre). Parameters for the selected step: family name, assembly, output directory, queue, and step-specific options. Fill in and click **Run on Cluster** or **Run locally**.

Log panel (bottom). Streams live output line-by-line: stage checkpoints, progress bars, and errors.

7 Connecting to an HPC cluster

1. Click **Settings** (gear icon) → **HPC Connection**.
2. Enter **Hostname**, **Username**, **SSH key path** (blank to use the SSH agent), **Scheduler** (LSF/Slurm/*Auto*), **Queue** / **partition**, and a writable **Work directory** (e.g. `/home/amodz/anno1/gameca`).
3. Click **Test connection**. A green tick confirms the scheduler was detected and the work directory is writable.

Duo / two-factor clusters: if your cluster requires MFA on every login, add a **ControlMaster** entry to `~/.ssh/config` so the app reuses one authenticated session:

```
Host hpclogin1.university.edu
  ControlMaster auto
  ControlPath ~/.ssh/cm-%r@%h:%p
  ControlPersist 8h
```

8 Running an analysis

8.1 Check the locus count first

Data prep → **RMSK locus count** (or **Dfam locus count**) verifies the family name and reports how many copies exist before you commit to a full job.

8.2 Single family — on the cluster

1. **Full pipeline** → **Batch job**.
2. Set **Family** (e.g. THE1A), **Assembly** (hg38/hg19/mm10/mm39), **Output directory**, and optionally **Notify email**.

3. Click **Run on Cluster**. GAMECA uploads the scripts, writes a `bsub/sbatch` job, and submits it. The job ID prints in the Log panel.
4. Monitor in **Results** → **File browser** or with `bjobs -w` on the cluster.
5. When done, **Results** → **Retrieve** rsyncs the output folder back to your Mac.

8.3 Single family — locally

Full pipeline → **Batch job**, fill in **Family** and **Assembly**, then **Run locally**. Output lands in `$HOME/gameca_results/<family>/`. Suitable for families up to ~5,000 copies on 16 GB RAM; larger families and ColabFold prediction should run on HPC.

8.4 Many families — batch submission

Submit many families as independent jobs with the bundled `submit_diego_families.py`. Each family produces *two* separate LSF jobs — a main job (core pipeline + Stage 11 except ColabFold) and a dependent fold job (ColabFold).

```
source ~/gameca_venv/bin/activate

python submit_diego_families.py \
    --outdir /home/$USER/gameca_results \
    --queue normal \
    --notify-email you@institution.edu

python submit_diego_families.py --dry-run # preview without submitting
```

Both jobs per family appear immediately in `bjobs`: `gam_<FAMILY>_main` starts when a slot frees; `gam_<FAMILY>_fold` stays PEND until its main job finishes, then starts automatically. Two emails are sent per family (one when the main job ends, one when the fold job ends).

9 Output reference

A run writes one folder per family. The tree below is the complete layout; the subsections that follow show *what is inside each file* so you can read the results without ever opening the app.

```
<outdir>/<family>/ e.g. gameca_results/thela/
  thela_sequences.csv flat list of every copy + sequence
  pipeline_diagnostic.log timestamped log of every stage
  pipeline_errors.log errors only (empty on a clean run)
  01_data/
    thela_with_sequences.csv copies + sequences, BEFORE clustering
    thela_clustered.csv master table: copies + cluster labels
  02_statistics/
    overall_statistics.txt whole-family length/GC summary
    cluster_summary.csv one row per cluster (size, GC, length)
    per_cluster/cluster_<c>_statistics.txt detailed stats per cluster
  03_clustering/
    clustering_visualization.html interactive UMAP scatter
    clustering_visualization_expr.html (if expression provided)
  04_alignments/
    alignment_stats.txt
```

```

05_consensus/
  the1a_consensus.fa one consensus per cluster + global
05_tfbs/
  cluster_tf_binding_counts.csv TF binding sites per cluster
  strand_tf_binding_counts.csv
  cluster_tf_binding_heatmap.html
  tf_binding_top_overlaps.html
06_primers/
  cluster_top5_primers.csv recommended primers per cluster
  selected_primers_summary.csv
07_visualizations/
  index.html DASHBOARD - start here, links all below
  cluster_distribution.html cluster sizes / composition
  sequence_characteristics.html length + GC distributions
  expression_heatmap.html (if expression provided)
  expression_comparison.html (if expression provided)
cluster_alignments/
  cluster_<c>_seqs.fa / _aligned.fa / _consensus.fa
  all_cluster_consensuses.fa
cialign_plots/
  index.html open in a browser to view alignments
motif_analysis/
  all_overlaps.tsv every copy x motif overlap
  overall_motif_counts.csv motif frequency table
  overall_top_motifs.png
  te_loci.bed, jaspar_indexed/ (intermediate - safe to ignore)
enrichment_results/
  cluster_<c>_enrichment.csv Fisher test per motif per cluster
  cluster_<c>_enrichment_annotated.csv
  enrichment_heatmap.png
go_annotations/
  gene_functions.csv TF -> gene -> GO lookups
  go_bp_top_terms.csv
  go_bp_top_terms.png
reports/ Stage 11 results (see below)
  stdout_analysis.log
  fig_*.pdf / *_per_copy.csv / *_report.txt / *_measured_values.tex
  fold/ ColabFold 3D structures (orfs.fa, *.pdb)
expression_plots/ expression + DE between clusters
  de_significant.csv, de_volcano.png, stage_profile.png, ...
jaspar/, tmp_motif/ (download cache + scratch - ignore)
bsub_main.out / bsub_fold.out live cluster job logs

```

Where to start: open `07_visualizations/index.html` in a browser — it is a dashboard that links every figure and table below. The subsections that follow walk through each output in the order the pipeline produces it, and explain not just *what* each file is but *how to read it*.

9.1 Sequences and statistics (the inputs to everything)

Before clustering, GAMECA writes the raw material it pulled from the genome. `01_data/<family>_with_sequences` (and the identical top-level `<family>_sequences.csv`) is one row per copy with its coordinates and extracted DNA, *before* any cluster labels are assigned. Use it to confirm the right family and

copy count were fetched. If this file is short or its **Seq** column is full of Ns, the genome fetch failed — fix that before trusting anything downstream.

02_statistics/overall_statistics.txt is a plain-text summary of the whole family: number of copies, and the distribution of sequence length and GC content. **per_cluster/cluster_<c>_statistics.txt** gives the same breakdown for each individual cluster. Read these to spot a family that is unexpectedly small, bimodal in length (often full-length vs. solo/truncated elements), or skewed in GC.

9.2 Clustering: the master table

01_data/<family>_clustered.csv is the central output: one row per genomic copy, identical to the sequences table but with clustering columns added. **Cluster** is the group it was sorted into; **chr/start/stop/strand** give its location; **Seq** is its DNA; **umap_x/umap_y** are the 2-D coordinates plotted in the interactive scatter.

```
Seq,Cluster,chr,start,stop,strand,te_name,umap_x,umap_y
tgttattagacgcg...,3,chr1,3031358,3031710,-,IAPLTR1a_Mm,4.21,-1.08
aatattcagttgtc...,1,chr2,9881204,9881559,+,IAPLTR1a_Mm,-2.66,3.94
```

How to read the clusters:

- Each **Cluster** value is a group of copies with similar sequence (*k*-mer) composition — typically corresponding to a subfamily, an age cohort, or a structural variant of the element.
- **Cluster = -1 is the noise group**: copies that did not fit any dense cluster (HDBSCAN's outliers). A large -1 group means the family is heterogeneous or the copies are short/divergent; it is normal to see some, but if *most* copies are -1 the family may be too diverse for tight clustering.
- A typical family yields a handful to a few dozen clusters. One or two dominant clusters plus several small ones is the usual shape.

02_statistics/cluster_summary.csv summarises each cluster (number of copies, mean length, GC content) — the quickest numeric overview of how the family split. The interactive UMAP scatter in **03_clustering/clustering_visualization.html** colours every copy by cluster: open it in a browser and hover any point for its coordinates and cluster. **Well-separated, compact blobs** mean clean subfamily structure; **one diffuse cloud** means the copies form a continuum rather than discrete groups. The **_expr** variant colours the same scatter by expression instead, when an expression matrix was supplied.

9.3 Alignment and consensus

cluster_alignments/cluster_<c>_consensus.fa is the “average” sequence of each cluster in FASTA format; the > line is the name and the lines below are the sequence. **all_cluster_consensuses.fa** bundles them all and is the input for downstream structure/divergence modules.

```
>IAP_cluster3_consensus
GGGGCTATTTTGGTTGGTGTGGACGTTTTTGTCTCGAAAATTAAGAGG
ATGCCAAAAGGTCCGATTGAAAAACAGTCTTTACATTGAAAATATCTA
```

`cialign_plots/index.html` is an HTML gallery of the CIAAlign before/after/markup images — the quickest way to judge how clean each cluster’s alignment is.

9.4 Motif analysis (`motif_analysis/`)

This stage scans every copy for JASPAR transcription-factor binding motifs. `all_overlaps.tsv` is the raw result: one line per (copy, motif) overlap, giving the copy coordinates, the motif name, its position, score, and strand — this is the table every downstream enrichment/TFBS step is built from.

`overall_motif_counts.csv` collapses that to a simple frequency table (how many times each motif was found across all copies), and `overall_top_motifs.png` plots the top 20.

```
Motif_name,count
CTCF,1840
SP1,1422
KLF4,1190
```

9.5 Enrichment (`enrichment_results/`)

Finding a motif is not the same as it being *characteristic* of a cluster. For each cluster, GAMECA runs a Fisher’s exact test asking “is this motif over-represented in this cluster versus the rest of the family?” `cluster_<c>_enrichment.csv` holds one row per motif, sorted by P_Value:

```
Motif,In_Cluster>Total_With_Motif,Cluster_Size>Total_Loci,Odds_Ratio,P_Value
CTCF,118,402,140,1485,6.91,3.2e-41
SP1,77,388,140,1485,3.10,1.1e-12
```

- `In_Cluster` — copies in this cluster with the motif.
- `Total_With_Motif` — copies family-wide with the motif.
- `Odds_Ratio` — how much more frequent the motif is in this cluster (> 1 = enriched).
- `P_Value` — Fisher’s exact significance; rows with `P_Value` below the threshold are the cluster’s signature TFs.

`cluster_<c>_enrichment_annotated.csv` adds the transcription-factor name, a one-line summary, and GO terms (see §9.7) to every significant motif. `enrichment_heatmap.png` shows clusters × enriched motifs at a glance.

9.6 TFBS analysis (`05_tfbs/`)

Where enrichment asks “which motifs are statistically special”, the TFBS stage simply *counts* binding sites so you can compare raw binding load across clusters. `cluster_tf_binding_counts.csv` has one row per (cluster, motif):

```
Cluster,Motif_name,Binding_Sites
0,CTCF,118
0,SP1,77
1,REST,54
```

strand_tf_binding_counts.csv breaks the same counts down by DNA strand. The two **.html** files are interactive Plotly views: **cluster_tf_binding_heatmap.html** (clusters $\times$ TFs, log-scaled) and **tf_binding_top_overlaps.html** (the 30 highest single overlaps). Open either in a browser.

9.7 GO annotation (go_annotations/)

This stage answers “what do the enriched transcription factors actually *do*?” Each significant motif is mapped to its gene via **mygene.info**, and **gene_functions.csv** records the function and Gene Ontology terms:

```
Gene,Symbol,Name,Summary,GO_BP,GO_MF,GO_CC
CTCF,CTCF,CCCTC-binding factor,"Chromatin insulator...",chromatin organization;...,DNA
binding;...,nucleus;...
```

go_bp_top_terms.csv ranks the most common Biological Process terms across all enriched TFs, and **go_bp_top_terms.png** (plus per-cluster **cluster_<c>_go_bp.png**) plots them.

```
GO_Term,Count
regulation of transcription,42
chromatin organization,28
```

If no motifs pass the significance threshold the stage writes a **GO_SKIPPED.txt** note instead of empty files — expected for small or highly divergent families.

9.8 Primers (06_primers/)

cluster_top5_primers.csv lists the recommended PCR primers per cluster. **coverage** is how many copies the primer matches; **total_expr** sums their expression; **rows_covered** names the copies. **selected_primers_summary.csv** is the global shortlist with the selection **strategy** used.

```
cluster,primer,coverage,total_expr,rows_covered
0,CCTAGCTATGCTGCCTTG,6,1119.19,"8,9,10,11,12,13"
```

How to choose a primer: prefer high **coverage** (amplifies many copies of the cluster) *and* high **total_expr** (those copies are actually expressed) if your goal is to target the active members. For a cluster-specific assay, pick a primer whose **rows_covered** are concentrated in one cluster rather than spread across the family.

9.9 The visual dashboard (07_visualizations/)

index.html is the recommended starting point for reading any run — open it in a browser and it embeds and links every figure the pipeline made. It pulls together four interactive Plotly pages, each also openable on its own:

File	What it shows
<code>cluster_distribution.html</code>	Bar/pie view of how many copies fell into each cluster — the visual version of <code>cluster_summary.csv</code> . Use it to see which subfamilies dominate.
<code>sequence_characteristics.html</code>	Distributions of copy length and GC content, split by cluster. A second length mode usually separates full-length from solo/truncated elements.
<code>expression_heatmap.html</code>	Copies $\times$ expression-stage heatmap (only written when an expression matrix is supplied). Bright rows are the actively transcribed copies.
<code>expression_comparison.html</code>	Expression compared <i>between</i> clusters, to see which sub-family carries the family’s transcriptional activity.

All four are interactive: hover for values, drag to zoom, double-click to reset. They need only a web browser — no other software.

9.10 Stage 11 standout analyses (reports/)

Stage 11 is a battery of 16 independent analyses. They share a consistent naming convention, so once you know it you can find any file by guessing:

- `fig_<analysis>_*.pdf/.png` — the figure(s).
- `<analysis>_per_copy.csv` — one row per genomic copy, adding that analysis’s columns. Join back to the master table (§9.2) by coordinates or TE_ID.
- `<analysis>_report.txt` — a plain-English summary of the headline numbers; needs no software and is the fastest read.
- `<analysis>_measured_values.tex` — the same numbers as L^AT_EX macros for the PDF report; ignore unless compiling it.

A typical `*_report.txt` looks like this:

```

GAMECA Phylogenetics / Divergence-Age --- Measured Values
Family: IAP
Copies analysed: 12577
Median est. age: 423.18 Myr
Intact (ORF>=thr): 3374
-- Top master/source element --
Name: chr8:4886476

```

The rest of this section documents every analysis and its exact output files, so any filename in `reports/` is searchable here.

1. Phylogenetics & divergence-age (phylo). Builds a tree, estimates each copy’s age from sequence divergence (a molecular clock), and flags the youngest intact “master” copies. **Files:** `fig_phylo_tree.pdf` (neighbour-joining tree), `fig_phylo_divergence.pdf` (divergence distribution), `fig_phylo_master.pdf` (candidate source elements); `phylo_per_copy.csv` (per copy: divergence, estimated age in Myr, ORF-intact flag); `phylo_report.txt`; `phylo_measured_values.tex`.

Read: low divergence = young/recently active; “intact” copies with the lowest divergence are the likely active source elements.

2. Consensus distance (consensus_distance). Measures how far each copy has drifted from its cluster consensus and from the global family consensus (CIAAlign-cleaned, Kimura-2P distance).

Files: `fig_cluster_consensus_distance.pdf`, `fig_global_consensus_distance.pdf`; `consensus_distance_per_copy.csv` (columns `TE_ID`, `cluster`, `dist_cluster`, `dist_global`). *Read:* small distances = copies that closely match the prototype; long tails = degraded or divergent copies.

3. CRISPR gRNA off-target scoring (grna). Scores candidate guide RNAs against the *whole* copy landscape — the real specificity problem for repeats. **Files:** `fig_grna_offtarget_pareto.pdf` (coverage-vs-specificity frontier), `fig_grna_offtarget_top.pdf`, `fig_grna_offtarget_mm.pdf` (mismatch tolerance); `grna_offtarget_guides.csv` (columns `guide`, `coverage`, `offtarget`, `cov_pct`, `on_target`, `specificity`, `pareto`); `grna_offtarget_report.txt`; `grna_offtarget_measured_values.tex`. *Read:* pick guides on the Pareto frontier (`pareto=True`) — high coverage, low offtarget.

4. 3' transduction (transduction). Detects copies that carried flanking 3' sequence with them when they mobilised, and groups them into mobilisation lineages. **Files:** `fig_transduction_groups.pdf` (shared-tail lineages), `fig_transduction_polya.pdf` (poly-A signal); `transduction_per_copy.csv` (per copy: transduced-tail group, poly-A presence); `transduction_report.txt`; `transduction_measured_values.tex`. *Read:* copies sharing a transduced tail descend from the same active source copy.

5. Antisense / bidirectional promoter (antisense). Scans both strands for promoter motifs and (with expression data) measures antisense:sense activity. **Files:** `fig_antisense_motifs.pdf`, `fig_antisense_expr.pdf`; `antisense_per_copy.csv` (columns include `sense_promoter_motifs`, `antisense_promoter_motifs`, `cpg_ratio`, `bidirectionality`); `antisense_report.txt`; `antisense_measured_values.tex`. *Read:* high bidirectionality marks copies that can drive transcription into neighbouring genes on either strand.

6. CTCF / TAD boundaries (ctcf_tad). Scans for CTCF binding motifs and measures each copy's distance to the nearest TAD (topological domain) boundary. **Files:** `fig_ctcf_overlap.pdf` (CTCF peak overlap), `fig_ctcf_tad_distance.pdf`; `ctcf_tad_per_copy.csv` (per copy: CTCF motif count, ChIP-peak overlap, distance to TAD boundary); `ctcf_tad_report.txt`; `ctcf_tad_measured_values.tex`. *Read:* copies at TAD boundaries with CTCF sites are candidates for re-shaping genome architecture.

7. Epigenetic overlay (epigenetic). Annotates each copy with ENCODE/Roadmap chromatin tracks (H3K9me3, ATAC, CpG, etc.) for the chosen cell line. **Files:** `fig_epigenetic_overlap.pdf`, `fig_epigenetic_heatmap.pdf` (copies $\times$ marks); `epigenetic_per_copy.csv` (per copy: signal per track); `epigenetic_report.txt`; `epigenetic_measured_values.tex`. *Read:* H3K9me3 = silenced/repressed; ATAC/open chromatin = potentially active.

8. Orthologous insertion presence (ortholog). Uses liftOver to test which copies are present at the same locus in other species (shared = ancestral; absent = lineage-specific). **Files:** `fig_ortholog_presence.pdf` (copies $\times$ species matrix), `fig_ortholog_summary.pdf`; `ortholog_per_copy.csv` (per copy: present/absent per species); `ortholog_report.txt`; `ortholog_measured_values.tex`. *Read:* a copy

present across many species predates their common ancestor; species-specific copies inserted more recently.

9. Multi-assembly liftover (multiassembly). Maps the family between assemblies (e.g. hg38 ↔ T2T-CHM13) to find copies one assembly resolves and another misses. **Files:** `fig_multiassembly_mapping.pdf`; `fig_multiassembly_coords.pdf`; `multiassembly_per_copy.csv` (per copy: mapped coordinates in each target assembly); `multiassembly_report.txt`; `multiassembly_measured_values.tex`. *Read:* copies that map only in T2T-CHM13 sit in pericentromeric/ heterochromatic regions hg38 leaves collapsed.

10. Consensus ORF structure prediction (fold). Folds the longest ORF of each cluster consensus with ColabFold (AlphaFold2). **Files:** `fig_fold_orf_map.pdf` (ORF positions), `fig_fold_plddt.pdf` (confidence per ORF), `fig_fold_summary.pdf`; `fold_report.txt`; `fold_measured_values.tex` and the `reports/fold/` sub-folder with the raw ColabFold output: `orfs.fa` (the protein sequences folded), one `*.pdb` per ORF (the 3-D structure — open in PyMOL, ChimeraX, or <https://molstar.org/viewer>), `*_plddt.png` (per-residue confidence; > 70 reliable, > 90 high), and `*_pae.png` (predicted aligned error; blue blocks = confidently-positioned domains). *Read:* a high-pLDDT structure means the consensus encodes a real, foldable protein domain — evidence the element was protein-coding.

11. Divergence landscape (divergence). Builds the classic “repeat landscape” — a histogram of copies by sequence divergence, the standard view of a family’s age structure. **Files:** `fig_repeat_landscape.pdf` (and `.png`), `fig_repeat_landscape_per_cluster.png`; `divergence_per_locus.csv` (per copy: % divergence from consensus), `divergence_stats.csv` (summary statistics); `repeat_landscape_values.tex`. *Read:* a peak at low divergence = a recent burst of activity; a broad spread = sustained or ancient activity.

12. LTR structural classification (ltr_struct). For LTR/ERV families, classifies each copy as full-length, solo-LTR, or truncated, and detects PBS/PPT replication motifs. **Files:** `fig_ltr_struct.png`; `ltr_struct_annotated.csv` (per copy: structure class, PBS/PPT, LTR identity), `ltr_struct_summary.csv` (counts of `n_full_length`, `n_solo_ltr`, `n_truncated`, `n_pbs`, `n_ppt`, `mean_ltr_identity`); `ltr_struct_values.tex`. *Read:* a high solo-LTR fraction indicates old elements that have recombined away their internal sequence. (Reports N/A for non-LTR families such as satellites.)

13. Subfamily resolution (subfamily). Resolves finer subfamily structure within the family and builds a subfamily tree. **Files:** `fig_subfamily_tree.png`, `fig_subfamily_divergence.png`; `subfamily_table.csv` (subfamily assignments + stats), `subfamily_tree.nwk` (Newick tree — open in FigTree or iTOL); `subfamily_values.tex`. *Read:* branch lengths show how diverged the subfamilies are from one another.

14. Benchmarking (benchmark). Records the pipeline’s per-stage runtime and scope for this family. **Files:** `fig_benchmark_steps.png` (per-stage timing); `benchmark_table.csv` (stage, seconds); `benchmark_values.tex`. *Read:* use it to see which stages dominate runtime when planning larger jobs.

15. Motif gain/loss (motif_gain). Identifies transcription-factor motifs each copy has *gained* or *lost* relative to the family consensus — regulatory innovation. **Files:** `fig_motif_gains_bar.png`, `fig_motif_gains_heatmap.png`; `motif_gain_per_copy.csv` (per copy: motifs gained/lost), `motif_gain_summary.csv` (most frequently gained motifs); `motif_gain_values.tex`. *Read:* a motif gained by many copies of one cluster points to cluster-specific regulatory rewiring.

16. Expression & differential expression (expression). Runs only when an expression matrix is present. **Writes to a separate expression_plots/ folder** at the family root (not `reports/`). **Files:** `boxplot_all.png`, `stage_profile.png`, `peak_stage_summary.png/.csv` (when each copy peaks), `chromosomal_expression.png/.csv` and `chromosomal_heatmap.png` (expression by location), `expression_stats.csv` (per-copy summary statistics), `primer_expression.png/.csv` and `primer_expression_heatmap.png` (expression of primer-targeted copies), and the differential-expression-between-clusters set: `de_volcano.png`, `de_pairwise.csv` (all pairwise tests), `de_significant.csv` (significant hits), `de_heatmap.png`, and `de_values.tex`. *Read:* `de_significant.csv` answers “which subfamily is differentially expressed, and at which stage” — the headline biological result when expression is available.

9.11 The compiled PDF report

`reports/standout_analysis.log` records each Stage 11 step with its exit code and timing — check it first if a figure is missing:

```
[phylo] ... ok (612s)
[grna] ... ok (94s)
[fold] ... FAILED rc=1 (12s) <- look here if a result is absent
```

Every figure and measured number can be stitched into a single publication-style PDF. From the output folder:

```
tectonic gameca_report.tex # or: pdflatex gameca_report.tex
```

9.12 Logs, caches, and intermediate files

The remaining files are not results, but knowing what they are tells you where to look when something goes wrong — and which files you can safely delete to reclaim space.

File / folder	Purpose
pipeline_diagnostic.log	Timestamped record of every stage as it ran. The first place to look if a run ended early or a folder is missing.
pipeline_errors.log	Errors only; empty on a clean run . A non-empty file points straight at what failed.
reports/standout_analysis.log	Per-module Stage 11 log with exit codes and timings (see §9.10).
bsub_main.out / bsub_fold.out	Live stdout of the cluster jobs — <code>tail -f</code> either to watch a running job.
motif_analysis/te_loci.bed, te_loci_tmp.bed	The TE coordinates in BED form, used internally for the JASPAR motif intersection. Intermediate.
motif_analysis/jaspar_indexed/, jaspar/	Cached, indexed JASPAR motif database — kept so re-runs skip the download. Safe to delete; it is rebuilt on demand.
tmp_motif/	Scratch space for motif scanning (e.g. <code>jaspar_norm6.bed</code>). Safe to delete after a run.
motif_analysis/README.txt, 05_tfbs/README.txt	Auto-written notes describing those folders' contents.
go_annotations/go_plot_status.txt	Records whether GO plots were generated.
*_measured_values.tex	L ^A T _E X macros consumed by <code>gameca_report.tex</code> ; ignore unless compiling the paper.

Status / failure markers. A few zero-byte-style marker files appear *only* when a stage is skipped or fails, and explain why in one line: `GO_SKIPPED.txt` (no motifs passed the enrichment threshold) and `MOTIF_ANALYSIS_FAILED.txt` (JASPAR data could not be fetched — usually a network/proxy issue). If you see one of these, the matching result folder will be empty by design.

10 Updates

Python scripts are refreshed silently from the latest commit on `main` every launch. No action required.

App binary When a new GitHub Release has a higher version, a banner appears on next launch. Click **Install update**; the new `.dmg` is downloaded, signature-verified, and installed. Relaunch when prompted.

To update manually, download the latest `.dmg` from Releases and repeat §4.

11 Troubleshooting

“GAMECA is damaged and cannot be opened.” macOS quarantine. Right-click `GAMECA.app` → **Open** → **Open** (one time). If it persists, **System Settings** → **Privacy & Security** → **Open Anyway**.

Blank screen or endless spinner. The Python sidecar failed to start. Confirm you downloaded

the correct architecture. Open **Console.app**, filter for “GAMECA”, and look for `ImportError / sidecar not found`.

Dependency install fails on first launch. Run it manually, then relaunch:

```
python3 -m venv --system-site-packages ~/gameca_venv
source ~/gameca_venv/bin/activate
pip install -r \
  /Applications/GAMECA.app/Contents/Resources/requirements.txt
```

HPC connection fails. Verify SSH works from Terminal (`ssh -i ~/.ssh/id_ed25519 user@host`). For Duo clusters add a **ControlMaster** entry (see §8).

Jobs stuck in pend. Normal when the queue is full or a fold job is waiting on its main job. Check slots with `bqueues normal`; inspect a specific job with `bjobs -l <JOBID>`.

GO stage produced only GO_SKIPPED.txt. No motifs passed the enrichment p-value threshold — expected for small or highly divergent families. Lower `-p-threshold` or inspect `enrichment_results/` directly.

Motif/TFBS folders are empty. JASPAR motif data could not be fetched (network/proxy). Check `echo $https_proxy`; GAMECA reads standard proxy env vars. A `MOTIF_ANALYSIS_FAILED.txt` marker explains why.

ColabFold hangs or crashes on HPC. Old host GCC. Use a Singularity image:

```
python run_stage11_all.py --only fold \
  --singularity-image /path/to/colabfold.sif \
  --no-gpu # omit if GPUs are available
```

Sequences are all “N” or empty. The UCSC API was unreachable from the compute node. Check the proxy as above, or supply a local genome FASTA via `-genome`.